

CSE 565 Software Unit Testing Framework Project

Chase Coleman

Course:	CSE 565 Software Verification and Validation
Project:	Software Unit Testing Framework Project
Algorithm:	Heapsort
Language:	Python
Unit Testing Framework:	Python unittest

1 Development of the Algorithm

For this project, I implemented the Heapsort algorithm in Python. The implementation defines a `heap` class. The user creates a heap object from an input list, calls `heapify()`, and then reads the sorted result from `h.arr`.

The algorithm works by repeatedly building a max heap over the unsorted part of the list. The largest value is placed at index 0, swapped to the end of the unsorted section, and then excluded from the next heap-building pass. This process continues until the list is sorted in ascending order.

1.1 Heapsort Source Code

```
"""Heapsort implementation for the CSE 565 unit testing project."""
class heap:
    def __init__(self, arr=[]):
        self.arr = list(arr)

    def swap(self, endex):
        if endex > 1:
            temp = self.arr[0]
            self.arr[0] = self.arr[endex-1]
            self.arr[endex-1] = temp

    def build_max_heap(self, i, endex):
        # build max heap from the array until endex
        if i < endex:
            l = 2 * i + 1
            r = 2 * i + 2
            self.build_max_heap(l, endex)
            self.build_max_heap(r, endex)
            if l < endex:
                if self.arr[l] > self.arr[i]:
                    temp = self.arr[i]
                    self.arr[i] = self.arr[l]
                    self.arr[l] = temp
```

```

        if r < endex:
            if self.arr[r] > self.arr[i]:
                temp = self.arr[i]
                self.arr[i] = self.arr[r]
                self.arr[r] = temp

    def heapify(self):
        for i in range(len(self.arr)):
            self.build_max_heap(0, len(self.arr)-i)
            self.swap(len(self.arr)-i)

if __name__ == "__main__":
    sample = [12, 11, 13, 5, 6, 7]
    h = heap(sample)
    h.heapify()
    print(h.arr)

```

```

CSE565 > Software Unit Testing Project > heap_sort.py > heap
1  """Heapsort implementation for the CSE 565 unit testing project."""
2  class heap:
3      def __init__(self, arr=[]):
4          self.arr = list(arr)
5
6
7      def swap(self, endex):
8          if endex > 1:
9              temp = self.arr[0]
10             self.arr[0] = self.arr[endex-1]
11             self.arr[endex-1] = temp
12
13
14     def build_max_heap(self, i, endex):
15         # build max heap from the array until endex
16         if i < endex:
17             l = 2 * i + 1
18             r = 2 * i + 2
19             self.build_max_heap(l, endex)
20             self.build_max_heap(r, endex)
21             if l < endex:
22                 if self.arr[l] > self.arr[i]:
23                     temp = self.arr[i]
24                     self.arr[i] = self.arr[l]
25                     self.arr[l] = temp
26             if r < endex:
27                 if self.arr[r] > self.arr[i]:
28                     temp = self.arr[i]
29                     self.arr[i] = self.arr[r]
30                     self.arr[r] = temp
31
32
33     def heapify(self):
34         for i in range(len(self.arr)):
35             self.build_max_heap(0, len(self.arr)-i)
36             self.swap(len(self.arr)-i)
37
38
39
40
41 if __name__ == "__main__":
42     sample = [12, 11, 13, 5, 6, 7]
43     h = heap(sample)
44     h.heapify()
45     print(h.arr)
46

```

2 Unit Testing Framework and AI Prompt Generation

I used Python's built-in `unittest` framework. I chose this framework because it comes with Python, requires no additional packages, and supports clear assertions for comparing the expected and actual output of the Heapsort algorithm.

A generative AI tool was used to create the first version of the unit tests. The prompt explained the interface of my implementation and asked the AI to generate tests for normal and edge-case inputs.

2.1 Prompt Used

I wrote a Python Heapsort implementation for a software unit testing project. The code defines a class named `heap`. To use it, create `h = heap(input_list)`, call `h.heapify()`, and then check `h.arr` for the sorted result.

Please generate Python `unittest` test cases for this Heapsort implementation. Cover normal input and edge cases, including an empty list, one-element list, already sorted list, reverse-sorted list, duplicate values, negative numbers, mixed positive and negative numbers, and all equal values.

CSE565 > Software Unit Testing Project > ai_prompts.md > * # AI Prompts For Unit Testing Project > * ## Execution Result 1

```

1  # AI Prompts For Unit Testing Project
2
3  ## Prompt 1
4
5  """text
6  I wrote a Python Heapsort implementation for a software unit testing project.
7  The code defines a class named heap. To use it, create h = heap(input_list),
8  call h.heapify(), and then check h.arr for the sorted result.
9
10 Please generate Python unittest test cases for this Heapsort implementation.
11 Cover normal input and edge cases, including an empty list, one-element list,
12 already sorted list, reverse-sorted list, duplicate values, negative numbers,
13 mixed positive and negative numbers, and all equal values.
14 """
15
16 ## AI-Generated Response 1
17
18 """python
19 import unittest
20
21 from heap_sort import heap
22
23
24 class TestHeapSortAI(unittest.TestCase):
25     def run_heap_sort(self, values):
26         h = heap(values)
27         h.heapify()
28         return h.arr
29
30     def test_empty_list(self):
31         self.assertEqual(self.run_heap_sort([]), [])
32
33     def test_single_element_list(self):
34         self.assertEqual(self.run_heap_sort([5]), [5])
35
36     def test_already_sorted_list(self):
37         self.assertEqual(self.run_heap_sort([1, 2, 3, 4, 5]), [1, 2, 3, 4, 5])
38
39     def test_reverse_sorted_list(self):
40         self.assertEqual(self.run_heap_sort([5, 4, 3, 2, 1]), [1, 2, 3, 4, 5])
41
42     def test_duplicate_values(self):
43         self.assertEqual(self.run_heap_sort([4, 1, 3, 4, 2, 1]), [1, 1, 2, 3, 4, 4])
44
45     def test_negative_numbers(self):
46         self.assertEqual(self.run_heap_sort([-3, -1, -7, -2]), [-7, -3, -2, -1])
47
48     def test_mixed_positive_and_negative_numbers(self):
49         self.assertEqual(self.run_heap_sort([3, -1, 0, -5, 8]), [-5, -1, 0, 3, 8])
50
51     def test_all_equal_values(self):
52         self.assertEqual(self.run_heap_sort([2, 2, 2, 2]), [2, 2, 2, 2])
53
54
55 if __name__ == "__main__":
56     unittest.main()
57 """
58
59 ## Execution Result 1
60
61 Summary:
62
63 - Test framework: Python `unittest`
64 - Test file: `test_heap_sort_ai.py`
65 - Tests run: 8
66 - Result: Passed
67 - Screenshot evidence: `screenshots/eight ai tests.png`
68
69 ![AI-generated unit tests passed](screenshots/eight%20ai%20tests.png)

```

3 AI-Generated Test Cases

The AI-generated unit tests were saved in `test_heap_sort_ai.py`. The test file contains eight tests covering empty input, a single-element list, an already sorted list, a reverse-sorted list, duplicates, negative numbers, mixed positive and negative numbers, and all-equal values.

3.1 AI-Generated Test Code

```
import unittest

from heap_sort import heap

class TestHeapSortAI(unittest.TestCase):
    def run_heap_sort(self, values):
        h = heap(values)
        h.heapify()
        return h.arr

    def test_empty_list(self):
        self.assertEqual(self.run_heap_sort([]), [])

    def test_single_element_list(self):
        self.assertEqual(self.run_heap_sort([5]), [5])

    def test_already_sorted_list(self):
        self.assertEqual(self.run_heap_sort([1, 2, 3, 4, 5]), [1, 2, 3, 4, 5])

    def test_reverse_sorted_list(self):
        self.assertEqual(self.run_heap_sort([5, 4, 3, 2, 1]), [1, 2, 3, 4, 5])

    def test_duplicate_values(self):
        self.assertEqual(self.run_heap_sort([4, 1, 3, 4, 2, 1]), [1, 1, 2, 3, 4, 4])

    def test_negative_numbers(self):
        self.assertEqual(self.run_heap_sort([-3, -1, -7, -2]), [-7, -3, -2, -1])

    def test_mixed_positive_and_negative_numbers(self):
        self.assertEqual(self.run_heap_sort([3, -1, 0, -5, 8]), [-5, -1, 0, 3, 8])

    def test_all_equal_values(self):
        self.assertEqual(self.run_heap_sort([2, 2, 2, 2]), [2, 2, 2, 2])

if __name__ == "__main__":
    unittest.main()
```

4 AI-Generated Test Execution

The AI-generated tests were executed with this command:

```
python3 -m unittest test_heap_sort_ai.py
```

The test run completed successfully.

- Test framework: Python `unittest`
- Test file: `test_heap_sort_ai.py`
- Tests run: 8
- Result: Passed

```
.....
-----
Ran 8 tests in 0.000s

OK
```

```
ASU > cd "CSE565/Software Unit Testing Project"
python3 -m unittest -v test_heap_sort_ai.py
test_all_equal_values (test_heap_sort_ai.TestHeapSortAI.test_all_equal_values) ... ok
test_already_sorted_list (test_heap_sort_ai.TestHeapSortAI.test_already_sorted_list) ... ok
test_duplicate_values (test_heap_sort_ai.TestHeapSortAI.test_duplicate_values) ... ok
test_empty_list (test_heap_sort_ai.TestHeapSortAI.test_empty_list) ... ok
test_mixed_positive_and_negative_numbers (test_heap_sort_ai.TestHeapSortAI.test_mixed_positive_and_negative_numbers) ... ok
test_negative_numbers (test_heap_sort_ai.TestHeapSortAI.test_negative_numbers) ... ok
test_reverse_sorted_list (test_heap_sort_ai.TestHeapSortAI.test_reverse_sorted_list) ... ok
test_single_element_list (test_heap_sort_ai.TestHeapSortAI.test_single_element_list) ... ok

-----
Ran 8 tests in 0.000s

OK
Software Unit Testing Project > █
```

Figure 3: AI-generated unit tests passing.

5 Assessment and Improvement of Test Cases

The AI-generated tests were a good starting point because they tested several basic sorting cases. However, the AI tests were mostly simple example-based tests. They did not check whether the original input list stayed unchanged, did not use randomized testing, and did not compare many different inputs against a trusted sorting function.

I improved the test suite in `test_heap_sort_student.py`. The student-improved tests kept the original coverage and added stronger cases.

- A test for preserving the original input list
- A larger reverse-sorted list
- Mixed float, negative, and duplicate values
- Randomized lists compared against Python's built-in `sorted` function
- The student-added float list and larger shuffled integer list tests

5.1 Student-Improved Test Code

```
import random
import unittest

from heap_sort import heap

class TestHeapSortStudent(unittest.TestCase):
    def run_heap_sort(self, values):
        h = heap(values)
        h.heapify()
        return h.arr

    def test_empty_list(self):
        self.assertEqual(self.run_heap_sort([]), [])

    def test_single_element_list(self):
        self.assertEqual(self.run_heap_sort([5]), [5])

    def test_already_sorted_list(self):
        self.assertEqual(self.run_heap_sort([1, 2, 3, 4, 5]), [1, 2, 3, 4, 5])

    def test_reverse_sorted_list(self):
        self.assertEqual(self.run_heap_sort([5, 4, 3, 2, 1]), [1, 2, 3, 4, 5])

    def test_duplicate_values(self):
        self.assertEqual(self.run_heap_sort([4, 1, 3, 4, 2, 1]), [1, 1, 2, 3, 4, 4])

    def test_negative_numbers(self):
        self.assertEqual(self.run_heap_sort([-3, -1, -7, -2]), [-7, -3, -2, -1])

    def test_mixed_positive_and_negative_numbers(self):
        self.assertEqual(self.run_heap_sort([3, -1, 0, -5, 8]), [-5, -1, 0, 3, 8])

    def test_all_equal_values(self):
        self.assertEqual(self.run_heap_sort([2, 2, 2, 2]), [2, 2, 2, 2])

    def test_float_list(self):
        self.assertEqual(self.run_heap_sort([3, 4, 1, 0.0]), [0.0, 1, 3, 4])

    def test_random_element_list(self):
        self.assertEqual(self.run_heap_sort([14, 12, 13, 15, 5, 6, 7, 9, 8, 10, 11, 4, 3, 2, 1]),
            , [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15])

    def test_original_input_is_not_changed(self):
        values = [4, 1, 3, 2]
        original = list(values)

        self.run_heap_sort(values)

        self.assertEqual(values, original)

    def test_mixed_float_negative_and_duplicate_values(self):
        values = [2.5, -1.0, 2.5, 0, -3.25, 8]
        self.assertEqual(self.run_heap_sort(values), [-3.25, -1.0, 0, 2.5, 2.5, 8])
```



```

def test_large_reverse_sorted_list(self):
    values = list(range(100, 0, -1))
    self.assertEqual(self.run_heap_sort(values), list(range(1, 101)))

def test_random_lists_match_python_sorted(self):
    random.seed(565)

    for _ in range(20):
        values = [random.randint(-100, 100) for _ in range(25)]
        self.assertEqual(self.run_heap_sort(values), sorted(values))

if __name__ == "__main__":
    unittest.main()

```

6 Student-Improved Test Execution

The improved student tests were executed with this command:

```
python3 -m unittest -v test_heap_sort_student.py
```

The improved test run completed successfully.

- Test framework: Python unittest
- Test file: test_heap_sort_student.py
- Tests run: 14
- Result: Passed

```

test_all_equal_values ... ok
test_already_sorted_list ... ok
test_duplicate_values ... ok
test_empty_list ... ok
test_float_list ... ok
test_large_reverse_sorted_list ... ok
test_mixed_float_negative_and_duplicate_values ... ok
test_mixed_positive_and_negative_numbers ... ok
test_negative_numbers ... ok
test_original_input_is_not_changed ... ok
test_random_element_list ... ok
test_random_lists_match_python_sorted ... ok
test_reverse_sorted_list ... ok
test_single_element_list ... ok

```

```
-----
Ran 14 tests in 0.001s
```

```
OK
```

```

Software Unit Testing Project > python3 -m unittest -v test_heap_sort_student.py
test_all_equal_values (test_heap_sort_student.TestHeapSortStudent.test_all_equal_values) ... ok
test_already_sorted_list (test_heap_sort_student.TestHeapSortStudent.test_already_sorted_list) ... ok
test_duplicate_values (test_heap_sort_student.TestHeapSortStudent.test_duplicate_values) ... ok
test_empty_list (test_heap_sort_student.TestHeapSortStudent.test_empty_list) ... ok
test_float_list (test_heap_sort_student.TestHeapSortStudent.test_float_list) ... ok
test_large_reverse_sorted_list (test_heap_sort_student.TestHeapSortStudent.test_large_reverse_sorted_list) ... ok
test_mixed_float_negative_and_duplicate_values (test_heap_sort_student.TestHeapSortStudent.test_mixed_float_negative_and_duplicate_values) ... ok
test_mixed_positive_and_negative_numbers (test_heap_sort_student.TestHeapSortStudent.test_mixed_positive_and_negative_numbers) ... ok
test_negative_numbers (test_heap_sort_student.TestHeapSortStudent.test_negative_numbers) ... ok
test_original_input_is_not_changed (test_heap_sort_student.TestHeapSortStudent.test_original_input_is_not_changed) ... ok
test_random_element_list (test_heap_sort_student.TestHeapSortStudent.test_random_element_list) ... ok
test_random_lists_match_python_sorted (test_heap_sort_student.TestHeapSortStudent.test_random_lists_match_python_sorted) ... ok
test_reverse_sorted_list (test_heap_sort_student.TestHeapSortStudent.test_reverse_sorted_list) ... ok
test_single_element_list (test_heap_sort_student.TestHeapSortStudent.test_single_element_list) ... ok

-----
Ran 14 tests in 0.001s

OK
Software Unit Testing Project >

```

Figure 4: Student-improved unit tests passing.

7 Assessment of the Generative AI Tool

The generative AI tool was useful for quickly creating a baseline unit test suite. It understood the interface of my Heapsort implementation, used the `unittest` framework correctly, and created tests for common cases such as empty lists, duplicate values, negative numbers, and reverse-sorted input.

The weakness of the AI-generated suite was that it did not go far beyond basic examples. It did not test whether the original input list was preserved, did not use randomized testing, and did not compare a variety of generated inputs against a known correct result. Because of that, human review was still necessary.

Overall, the AI tool was effective as a starting point, but it did not replace the need for student judgment. The AI-generated tests helped begin the testing process, while the student-improved tests gave stronger confidence that the Heapsort implementation behaves correctly.

8 Conclusion

This project implemented the Heapsort algorithm, generated unit tests using a generative AI tool, executed the AI-generated tests, evaluated their strengths and weaknesses, and improved the test suite with additional student-created cases. The AI-generated test suite passed all 8 tests, and the student-improved test suite passed all 14 tests.